

Resolução do Mundo dos Blocos de Tamanho Variável via Satisfatibilidade Booleana (SAT)

Davi T. Moraes¹, Sebastian L. Guedes¹, Vinicius O. Souza¹

¹ Instituto de Computação – Universidade Federal do Amazonas (UFAM)
Caixa Postal 2.432 – 69.080-900 – Manaus – AM – Brasil

{sebastian.guedes, davi.moraes, vinicius.souza}@icomp.ufam.edu.br

Abstract. *This paper presents the formal modeling and practical implementation of the Variable-Sized Blocks World planning problem using Boolean Satisfiability (SAT). By extending the classic Blocks World to accommodate blocks of varying length, multi-slot occupancy, bridges (blocks supported by multiple blocks), and physical stability constraints, we formulate the domain into Conjunctive Normal Form (CNF). The propositional logic encoding enables modern SAT solvers, such as miniSAT, to compute optimal sequence plans.*

Resumo. *Este artigo apresenta a modelagem formal e a implementação prática do problema de planejamento no Mundo dos Blocos de Tamanho Variável utilizando Satisfatibilidade Booleana (SAT). Ao estender o Mundo dos Blocos clássico para acomodar objetos de comprimentos variados, ocupação de múltiplos slots na mesa, pontes e restrições mecânicas de estabilidade, o domínio é traduzido em Forma Normal Conjuntiva (CNF). A codificação proposicional permite que resolvedores SAT modernos, como o miniSAT, encontrem planos ótimos de movimentação.*

1. Introdução e Objetivos

O **Mundo dos Blocos (Blocks World)** é um dos domínios clássicos mais emblemáticos na área de Inteligência Artificial para o estudo de planejamento automático. No modelo tradicional, todos os blocos possuem dimensão unitária e a relação de empilhamento é simples (um bloco diretamente sobre o outro ou sobre a mesa). Este trabalho, baseado em [1, 2], estende o modelo clássico para o **Mundo dos Blocos de Tamanho Variável**, onde os objetos possuem comprimentos variados, $\ell(b) \in \{1, 2, 3\}$, a mesa é dividida em *slots* unitários e os blocos podem formar pontes ou exigir apoios múltiplos para garantir a estabilidade mecânica.

1.1. Objetivos

- **Formalização Proposicional:** Representar formalmente um domínio com restrições espaciais e mecânicas usando Lógica Proposicional.
- **Codificação em CNF:** Mapear predicados de estado, ações qualitativas, axiomas de *frame* e restrições de estabilidade para cláusulas em Forma Normal Conjuntiva (CNF) no padrão DIMACS.
- **Planejamento via SAT Solver:** Utilizar o resolvidor *miniSAT* [3] para encontrar planos de horizonte mínimo T .
- **Reconstrução Simbólica:** Desenvolver um mecanismo de tradução (`.map`) para converter as atribuições numéricas booleanas retornadas pelo resolvidor de volta para sequências de ações em linguagem natural.

2. Descrição Formal do Domínio

A descrição segue o manual do caso simples, com o tempo t explícito (equivalente ao cálculo de situações, com s_t no lugar de t). As cláusulas CNF da Seção 3 são a tradução proposicional dos axiomas aqui apresentados.

2.1. Sintaxe do Domínio

Blocos e mesa. Seja $B = \{a, b, c, d\}$ o conjunto de blocos e $L = B \cup \{T\}$ o conjunto de apoios, onde T é a mesa (que pode conter vários blocos).

Comprimentos. A função $\ell : B \rightarrow \mathbb{N}$ dá o comprimento de cada bloco, em unidades de comprimento (uc): $\ell(a) = \ell(b) = 1$, $\ell(c) = 2$ e $\ell(d) = 3$.

Mesa, pontos e slots. O eixo é o conjunto de pontos $X = \{0, 1, \dots, 6\}$. Os **slots** são os intervalos unitários $s_i = [i, i + 1]$, com $i \in \{0, \dots, 5\}$. São 6 slots, e não 7: os números $0, \dots, 6$ das figuras são *pontos*. Os níveis de empilhamento são $l \in \{0, 1, 2, 3\}$, onde $l = 0$ é a mesa.

Posição horizontal. Um bloco b que começa no ponto p ocupa $[p, p + \ell(b)]$ e cobre os slots

$$\text{span}(b, p) = \{p, p + 1, \dots, p + \ell(b) - 1\}. \quad (1)$$

As posições válidas são $p \in \{0, \dots, 6 - \ell(b)\}$ (Tabela 1).

Tabela 1. Posições válidas por bloco.

Bloco	Posições válidas
a, b	$\{0, 1, 2, 3, 4, 5\}$
c	$\{0, 1, 2, 3, 4\}$
d	$\{0, 1, 2, 3\}$

2.2. Predicados de Estado (primitivos)

- $\text{At}(b, p, t)$: o bloco b começa no ponto p no instante t .
- $\text{Lev}(b, l, t)$: o bloco b está no nível l no instante t .

O estado do mundo em t é determinado por At e Lev de cada bloco. Todo o resto é derivado.

2.3. Relações Derivadas

Cobertura e ocupação de slots:

$$\text{Cov}(b, i, l, t) \leftrightarrow \exists p [\text{At}(b, p, t) \wedge \text{Lev}(b, l, t) \wedge i \in \text{span}(b, p)] \quad (2)$$

$$\text{Occ}(i, l, t) \leftrightarrow \exists b \text{Cov}(b, i, l, t) \quad (3)$$

Sobreposição de spans:

$$\text{Overlap}(b, p, y, q) \leftrightarrow \text{span}(b, p) \cap \text{span}(y, q) \neq \emptyset \quad (4)$$

Apoio (On). O bloco b está apoiado em y :

$$\text{On}(b, \top, t) \leftrightarrow \text{Lev}(b, 0, t) \quad (5)$$

$$\text{On}(b, y, t) \leftrightarrow \exists p, q, l [\text{At}(b, p, t) \wedge \text{At}(y, q, t) \wedge \text{Lev}(b, l, t) \wedge \text{Lev}(y, l-1, t) \wedge \text{Overlap}(b, p, y, q)] \quad (6)$$

Um bloco pode ter **múltiplos apoios** (a ponte). Por exemplo, em S_0 da Situação 1, $\text{On}(d, a, 0) \wedge \text{On}(d, b, 0)$, com o slot s_4 vazio sob d .

Topo livre. O bloco b não tem nada em cima:

$$\text{Clear}(b, t) \leftrightarrow \neg \exists i, l [\text{Cov}(b, i, l, t) \wedge \text{Occ}(i, l+1, t)] \quad (7)$$

Estabilidade. O bloco b em (p, l) é estável quando está na mesa ou tem pelo menos $\lceil \ell(b)/2 \rceil$ slots apoiados no nível abaixo:

$$\text{Stable}(b, t) \leftrightarrow \text{Lev}(b, 0, t) \vee \exists p, l [\text{At}(b, p, t) \wedge \text{Lev}(b, l, t) \wedge |\{i \in \text{span}(b, p) : \text{Occ}(i, l-1, t)\}| \geq \lceil \ell(b)/2 \rceil] \quad (8)$$

Na prática, a , b e c precisam de 1 slot de apoio, e d precisa de 2. Assim, d pode ficar em ponte sobre dois blocos separados, mas não apoiado em um único ponto.

2.4. Restrições de Estado (axiomas do domínio)

Valem em todo instante t :

(V1) Unicidade: $\text{At}(b, p, t) \wedge \text{At}(b, p', t) \rightarrow p = p'$ e $\text{Lev}(b, l, t) \wedge \text{Lev}(b, l', t) \rightarrow l = l'$ (e cada bloco tem posição e nível).

(V2) Limites: $\text{At}(b, p, t) \rightarrow 0 \leq p \leq 6 - \ell(b)$ e $\text{Lev}(b, l, t) \rightarrow 0 \leq l \leq 3$.

(V3) Exclusão horizontal: $b_1 \neq b_2 \wedge \text{Cov}(b_1, i, l, t) \rightarrow \neg \text{Cov}(b_2, i, l, t)$.

(V4) Estabilidade: $\text{Stable}(b, t)$ para todo bloco b .

2.5. Ação

A única ação do domínio é $\text{move}(b, y, p, t)$: mover o bloco b para cima do apoio y ($y \in L \setminus \{b\}$), começando no ponto p , no instante t . A ação é **qualitativa**: em vez de “para o nível 2, posição 3”, diz-se “para cima de c , começando em 0”. O nível de destino é consequência de y :

$$l' = \begin{cases} 0 & \text{se } y = \top, \\ \text{Lev}(y, t) + 1 & \text{se } y \in B. \end{cases} \quad (9)$$

2.6. Pré-condições e Efeitos (STRIPS)

Seja (p_0, l_0) a posição atual de b em t . A Tabela 2 resume pré-condições e efeitos.

Além disso, o estado em $t + 1$ deve satisfazer (V3) e (V4). Isso garante, por exemplo, que b não pousa instável e que nenhum outro bloco fica sem apoio quando b sai.

Diferença para o caso simples. Não se exige $\text{Clear}(y)$ do destino. A condição é que os **slots** de destino estejam livres (P4). Com a regra “topo de y livre”, só caberia um bloco por apoio, e estados como S_{f1} (com a e b lado a lado sobre d) seriam inalcançáveis.

Tabela 2. Pré-condições e efeitos de $\text{move}(b, y, p, t)$.

Pré-condições	Efeitos
(P1) $\text{Clear}(b, t)$	Add: $\text{At}(b, p, t+1)$
(P2) $0 \leq p \leq 6 - \ell(b)$ e $l' \leq 3$	Add: $\text{Lev}(b, l', t+1)$
(P3) se $y \in B$: $\exists q [\text{At}(y, q, t) \wedge \text{Overlap}(b, p, y, q)]$	Delete: $\text{At}(b, p_0, t+1)$, se $p_0 \neq p$
(P4) destino livre: $\forall i \in \text{span}(b, p), \forall z \neq b : \neg \text{Cov}(z, i, l', t)$	Delete: $\text{Lev}(b, l_0, t+1)$, se $l_0 \neq l'$
(P5) não é no-op: $\neg(\text{At}(b, p, t) \wedge \text{Lev}(b, l', t))$	Frame: os demais blocos mantêm At e Lev

2.6.1. Elementos novos e as ações associadas

Para cada elemento acrescentado em relação ao caso simples, a Tabela 3 indica o que a ação $\text{move}(b, y, p, t)$ adiciona ou remove.

Tabela 3. Elementos novos e efeito da ação.

Elemento novo	Natureza	Efeito da ação
$\ell(x)$, span	constante / função	nenhum (usados em Cov e Stable)
$\text{At}(b, p, t)$	primitivo	add $\text{At}(b, p, t+1)$, delete $\text{At}(b, p_0, t+1)$
$\text{Lev}(b, l, t)$	primitivo	add $\text{Lev}(b, l', t+1)$, delete $\text{Lev}(b, l_0, t+1)$
Cov, Occ	derivado	add nos slots de $\text{span}(b, p)$ em l' , delete nos slots de $\text{span}(b, p_0)$ em l_0
$\text{On}(b, \cdot)$	derivado	add $\text{On}(b, y')$ para todo y' no nível $l'-1$ que sobrepe b (podem ser vários), delete os apoios antigos
$\text{Clear}(y)$	derivado	delete nos blocos sob o novo span de b ; add nos blocos que ficam sem nada em cima depois que b sai
Stable	restrição	a ação só é aplicável se o estado em $t+1$ for estável para todos os blocos
espaços vazios	$\neg \text{Occ}(i, l, t)$	passam a ocupados no destino e a livres na origem

2.7. Configuração Inicial e Meta

Dada uma configuração inicial S_0 e uma meta G , procura-se uma sequência de T ações move que leve S_0 a um estado que satisfaça G .

- **Estado inicial:** para cada bloco b , $\text{At}(b, p_0, 0) \wedge \text{Lev}(b, l_0, 0)$.
- **Meta:** para cada bloco b , $\text{At}(b, p_G, T) \wedge \text{Lev}(b, l_G, T)$. A meta pode ser parcial (apenas alguns blocos).

Os estados das Situações 1, 2 e 3 estão na Seção 4.

2.8. Ordem Parcial

Duas metas (ou ações) φ_1 e φ_2 podem ter a ordem parcial $\varphi_1 \prec_P \varphi_2$, que significa “ φ_1 deve ocorrer antes de φ_2 ”:

$$\varphi_1 \prec_P \varphi_2 \equiv \forall t [\varphi_2(t) \wedge \neg \varphi_2(t-1) \rightarrow \exists t' < t \varphi_1(t')] \quad (10)$$

Ou seja, φ_2 só passa a valer depois de φ_1 já ter valido. Metas sem ordem entre si podem ser atingidas em qualquer sequência (daí “parcial”). Exemplo na Situação 1, S_{f4} : $\text{On}(d, T) \prec_P \text{On}(a, c)$ é possível, mas a ordem inversa não é, porque d precisa sair de cima de c antes.

2.9. Exemplo: aplicando $\text{move}(d, c, 0, 0)$ em S_0 (Situação 1)

Estado: $c(0, 0)$, $a(3, 0)$, $b(5, 0)$, $d(3, 1)$, onde $x(p, l)$ denota posição e nível. Então $l' = \text{Lev}(c, 0) + 1 = 1$. A Tabela 4 verifica cada condição.

Tabela 4. Verificação de $\text{move}(d, c, 0, 0)$ em S_0 .

Condição	Verificação
(P1) $\text{Clear}(d, 0)$	nada sobre d
(P2) $0 \leq 0 \leq 3, l' = 1 \leq 3$	satisfeita
(P3) $\text{Overlap}(d, 0, c, 0)$	$\{0, 1, 2\} \cap \{0, 1\} \neq \emptyset$
(P4) slots 0, 1, 2 livres no nível 1	só d ocupava o nível 1, e d é o bloco que se move
(P5) não é no-op	d estava em $(3, 1)$ e vai para $(0, 1)$
(V4) $\text{Stable}(d)$ em $t+1$	slots 0 e 1 apoiados por c : $2 \geq \lceil 3/2 \rceil = 2$

Efeitos: $\text{At}(d, 0, 1)$, $\text{Lev}(d, 1, 1)$ e a remoção de $\text{At}(d, 3, \cdot)$. Derivados: $\neg \text{Clear}(c, 1)$, e $\text{Clear}(a, 1)$ e $\text{Clear}(b, 1)$ passam a valer, pois d saiu de cima deles.

2.10. Resumo das diferenças para o caso simples

Tabela 5. Caso simples versus tamanho variável.

Caso simples	Tamanho variável
blocos de tamanho 1	$\ell(b) \in \{1, 2, 3\}$
posição = o que está embaixo (on)	posição horizontal (At) e nível (Lev)
um bloco sobre cada bloco	vários blocos lado a lado sobre o mesmo apoio
um único apoio	apoio múltiplo (ponte)
sem espaços vazios	slots vazios e movimento lateral
$\text{move}(x, y, z)$, de y para z	$\text{move}(b, y, p)$, qualitativa
pré-condição $\text{Clear}(z)$ do destino	destino livre por slot (P4)
sem estabilidade	estabilidade ($\lceil \ell/2 \rceil$ slots)

3. Codificação CNF

3.1. Variáveis Proposicionais

Para cada passo $t \in \{0, 1, \dots, T\}$ e cada bloco $b \in B$, criamos as variáveis booleanas:

- $\text{at}(b, p, t)$, para $p \in [0, 6 - \ell(b)]$ (o bloco b começa no ponto p);
- $\text{lev}(b, l, t)$, para $l \in [0, 3]$ (o bloco b está no nível l);
- $\text{clr}(b, t)$ (o topo de b está livre);
- $\text{mv}(b, y, p, t)$, para $t < T$, $y \in L \setminus \{b\}$, $p \in [0, 6 - \ell(b)]$ (mover b para cima de y , começando em p).

Aqui $L = B \cup \{T\}$ (a mesa pode ter vários blocos). Como $\ell(a) = \ell(b) = 1$, $\ell(c) = 2$ e $\ell(d) = 3$, há $\sum_b (7 - \ell(b)) = 6 + 6 + 5 + 4 = 21$ pares (bloco, posição).

Para escrever as cláusulas, usamos três variáveis **auxiliares**, que não fazem parte do domínio:

- $\text{cov}(b, i, l, t)$: o bloco b cobre o slot i no nível l ;
- $\text{occ}(i, l, t)$: algum bloco cobre o slot i no nível l ;
- $\text{blk}(b, i, l, t)$: b cobre i no nível l e há algo cobrindo i no nível $l + 1$.

O número total de variáveis é

$$|V| = \underbrace{21(T+1)}_{\text{at}} + \underbrace{16(T+1)}_{\text{lev}} + \underbrace{4(T+1)}_{\text{clr}} + \underbrace{84T}_{\text{mv}} + \underbrace{96(T+1)}_{\text{cov}} + \underbrace{96(T+1)}_{\text{blk}} + \underbrace{24(T+1)}_{\text{occ}} = 341T + 257. \quad (11)$$

Para a Situação 1 (4 blocos, $T = 4$): $|V| = 105 + 80 + 20 + 336 + 480 + 480 + 120 = 1621$. As 541 primeiras são as variáveis do domínio (as que vão para o arquivo `.map`), e as 1080 restantes são auxiliares.

A relação $\text{on}(b, y, t)$ **não é variável**: ela é derivada depois (Seção 3.3).

3.2. Regras de Codificação (Grupos de Cláusulas)

Notação: $\text{span}(b, p) = \{p, \dots, p + \ell(b) - 1\}$ são os slots cobertos por b quando começa em p . A numeração [1] a [12] é a mesma dos comentários do código.

[1] Estado inicial. Para cada bloco b com posição inicial (p_0, l_0) : $\text{at}(b, p_0, 0) \wedge \text{lev}(b, l_0, 0)$.

[2] Meta. Para cada bloco b com posição final (p_G, l_G) : $\text{at}(b, p_G, T) \wedge \text{lev}(b, l_G, T)$.

[3] Cada bloco em exatamente uma posição. Para todo b e t , e todo par $p_1 \neq p_2$:

$$\bigvee_p \text{at}(b, p, t) \quad \neg \text{at}(b, p_1, t) \vee \neg \text{at}(b, p_2, t)$$

[4] Cada bloco em exatamente um nível. Para todo b e t , e todo par $l_1 \neq l_2$:

$$\bigvee_l \text{lev}(b, l, t) \quad \neg \text{lev}(b, l_1, t) \vee \neg \text{lev}(b, l_2, t)$$

[Aux] Definição de cov e occ. Para todo b, i, l, t :

$$\begin{aligned} \neg \text{at}(b, p, t) \vee \neg \text{lev}(b, l, t) \vee \text{cov}(b, i, l, t) & \quad \forall p : i \in \text{span}(b, p) \\ \neg \text{cov}(b, i, l, t) \vee \text{lev}(b, l, t) & \quad \neg \text{cov}(b, i, l, t) \vee \bigvee_{p: i \in \text{span}(b, p)} \text{at}(b, p, t) \\ \neg \text{cov}(b, i, l, t) \vee \text{occ}(i, l, t) & \quad \neg \text{occ}(i, l, t) \vee \bigvee_b \text{cov}(b, i, l, t) \end{aligned}$$

[5] Exclusão horizontal. Dois blocos distintos no mesmo nível não podem compartilhar um slot. Para todo slot i , nível l , instante t e todo par $b_1 \neq b_2$:

$$\neg \text{cov}(b_1, i, l, t) \vee \neg \text{cov}(b_2, i, l, t)$$

[6] Estabilidade. Um bloco b em (p, l) com $l > 0$ precisa de pelo menos $k = \lceil \ell(b)/2 \rceil$ slots apoiados no nível $l-1$. “Pelo menos k de ℓ ” equivale a “todo subconjunto de $\ell-k+1$ slots tem algum slot apoiado”. Para todo $S' \subseteq \text{span}(b, p)$ com $|S'| = \ell(b) - k + 1$:

$$\neg \text{at}(b, p, t) \vee \neg \text{lev}(b, l, t) \vee \bigvee_{i \in S'} \text{occ}(i, l-1, t)$$

Blocos de tamanho 1 e 2 precisam de 1 slot de apoio, e o bloco d (tamanho 3) precisa de 2. Por isso d pode fazer ponte sobre dois blocos separados por um slot vazio, mas não pode ficar apoiado em um único ponto.

[7] Definição de clr. $\text{clr}(b, t)$ vale se, e somente se, nenhum slot coberto por b tem algo no nível acima:

$$\text{clr}(b, t) \leftrightarrow \neg \bigvee_{i, l} \text{blk}(b, i, l, t)$$

que se decompõe em

$$\neg \text{clr}(b, t) \vee \neg \text{blk}(b, i, l, t) \quad \text{clr}(b, t) \vee \bigvee_{i, l} \text{blk}(b, i, l, t)$$

com a definição de blk:

$$\begin{aligned} \neg \text{cov}(b, i, l, t) \vee \neg \text{occ}(i, l+1, t) \vee \text{blk}(b, i, l, t) \\ \neg \text{blk}(b, i, l, t) \vee \text{cov}(b, i, l, t) \quad \neg \text{blk}(b, i, l, t) \vee \text{occ}(i, l+1, t) \quad \neg \text{blk}(b, i, 3, t) \end{aligned}$$

[8] Pré-condições das ações. Para cada $\text{mv}(b, y, p, t)$:

$$\neg \text{mv}(b, y, p, t) \vee \text{clr}(b, t) \quad (\text{topo de } b \text{ livre})$$

Se $y \in B$, o span de b deve sobrepor o de y (para todo q com $\text{span}(y, q) \cap \text{span}(b, p) = \emptyset$):

$$\neg \text{mv}(b, y, p, t) \vee \neg \text{at}(y, q, t)$$

Os slots de destino devem estar livres (para todo $z \neq b$ e $i \in \text{span}(b, p)$):

$$\begin{aligned} y = \top : & \quad \neg \text{mv}(b, y, p, t) \vee \neg \text{cov}(z, i, 0, t) \\ y \in B : & \quad \neg \text{mv}(b, y, p, t) \vee \neg \text{lev}(y, l, t) \vee \neg \text{cov}(z, i, l+1, t) \quad \forall l \end{aligned}$$

A ação também não pode ser um no-op (o destino difere da posição atual) nem empilhar acima do nível 3. Diferente do caso simples, **não se exige** $\text{clr}(y)$ para o destino: basta que os slots de destino estejam livres, o que permite vários blocos lado a lado sobre o mesmo apoio (como a e b sobre d em S_{f1}).

[9] Efeitos das ações. Para cada $\text{mv}(b, y, p, t)$:

$$\begin{aligned} & \neg \text{mv}(b, y, p, t) \vee \text{at}(b, p, t+1) \\ y = \top : & \quad \neg \text{mv}(b, y, p, t) \vee \text{lev}(b, 0, t+1) \\ y \in B : & \quad \neg \text{mv}(b, y, p, t) \vee \neg \text{lev}(y, l, t) \vee \text{lev}(b, l+1, t+1) \quad \forall l \end{aligned}$$

A validade do estado resultante (sem sobreposição e estável) é garantida pelos grupos [5] e [6] aplicados em $t + 1$. Isso vale também para blocos que perdem apoio quando b sai.

[10] Axiomas de frame. O que não muda permanece igual. Para todo b, p, l e $t < T$:

$$\begin{aligned} & \neg \text{at}(b, p, t) \vee \text{at}(b, p, t+1) \vee \bigvee_{y, p'} \text{mv}(b, y, p', t) \\ & \neg \text{lev}(b, l, t) \vee \text{lev}(b, l, t+1) \vee \bigvee_{y, p'} \text{mv}(b, y, p', t) \end{aligned}$$

A variável clr não precisa de frame axiom, porque o grupo [7] a define exatamente em cada instante.

[11] No máximo uma ação por passo. Para todo $t < T$ e todo par de ações distintas $a_1 \neq a_2$:

$$\neg \text{mv}_{a_1}(t) \vee \neg \text{mv}_{a_2}(t)$$

[12] Ordem parcial (opcional). Seja $g(b, p, l, t) \leftrightarrow \text{at}(b, p, t) \wedge \text{lev}(b, l, t)$. Para metas $\varphi_1 \prec_P \varphi_2$ e todo $t \geq 1$:

$$\neg g_2(t) \vee g_2(t-1) \vee g_1(0) \vee \dots \vee g_1(t-1)$$

ou seja, φ_2 só passa a valer depois de φ_1 já ter valido.

3.3. A Relação on (derivada, não codificada)

Não criamos variáveis para $\text{on}(b, y, t)$. Ela é derivada a partir de at e lev :

$$\text{on}(b, \top, t) \leftrightarrow \text{lev}(b, 0, t)$$

$$\text{on}(b, y, t) \leftrightarrow \exists p, q, l. \text{at}(b, p, t) \wedge \text{at}(y, q, t) \wedge \text{lev}(b, l, t) \wedge \text{lev}(y, l-1, t) \wedge \text{Overlap}(b, p, y, q)$$

Isso permite o apoio múltiplo: $\text{on}(d, a, t) \wedge \text{on}(d, b, t)$ descreve a ponte. A reconstrução é feita pelo `interpretar.py`.

3.4. Tamanho da Codificação

Para os cenários com os 4 blocos totalmente especificados no estado inicial e na meta,

$$|V| = 341T + 257, \quad |C| = 6491T + 1113. \quad (12)$$

Tabela 6. Tamanho da codificação em função de T .

T	Variáveis	Cláusulas
2	939	14.095
4	1.621	27.077
6	2.303	40.059
9	3.326	59.532

O crescimento é linear em T . A fórmula de $|C|$ foi obtida a partir dos tamanhos gerados e confere com os sete horizontes testados ($T = 2, 3, 4, 5, 6, 8, 9$). Ela muda se a meta for parcial ou se `ORDEM_PARCIAL` tiver pares. Para provar que um plano é ótimo, executa-se o SAT solver para $T = 0, 1, 2, \dots$ até obter `SATISFIABLE`: o primeiro T que satisfaz é o comprimento mínimo do plano.

4. Exemplos dos Três Cenários Codificados

Em cada cenário são dados a representação formal, as relações on e as cláusulas unitárias (DIMACS) que fixam estado inicial e meta.

4.1. Situação 1

4.1.1. Estado inicial S_0

Cenário	T	Variáveis	Cláusulas (gerador)	Resultado no minisat
Sit. 1: $S_0 \rightarrow S_{f1}$	8	2.985	53.041	SAT
Sit. 1: $S_0 \rightarrow S_{f2}$	9	3.326	59.532	SAT
Sit. 1: $S_0 \rightarrow S_{f3}$	3	1.280	20.586	SAT
Sit. 1: $S_0 \rightarrow S_{f4}$	4	1.621	27.077	SAT
Sit. 2: $S_0 \rightarrow S_5$	5	1.962	33.568	SAT
Sit. 3: $S_0 \rightarrow S_7$	6	2.303	40.059	SAT

Figura 1. Diagrama da Situação 1 – Estado Inicial S_0 : bloco d em ponte sobre a e b .

$$\begin{array}{ll} \text{at}(c, 0, 0) \wedge \text{lev}(c, 0, 0) & \text{at}(a, 3, 0) \wedge \text{lev}(a, 0, 0) \\ \text{at}(b, 5, 0) \wedge \text{lev}(b, 0, 0) & \text{at}(d, 3, 0) \wedge \text{lev}(d, 1, 0) \end{array}$$

Relações on em $t = 0$: $\text{on}(c, T, 0)$, $\text{on}(a, T, 0)$, $\text{on}(b, T, 0)$, $\text{on}(d, a, 0) \wedge \text{on}(d, b, 0)$. Note que d está em *ponte* sobre a e b , com o slot s_4 vazio (mas bloqueado para receber um bloco, por estar sob d).

Cláusulas unitárias: 119 0, 124 0, 4 0, 7 0, 65 0, 66 0, 180 0, 182 0.

4.1.2. Meta S_{f_4} ($T = 4$)

$$\begin{array}{ll} \text{at}(c, 0, T) \wedge \text{lev}(c, 0, T) & \text{at}(a, 0, T) \wedge \text{lev}(a, 1, T) \\ \text{at}(d, 2, T) \wedge \text{lev}(d, 0, T) & \text{at}(b, 5, T) \wedge \text{lev}(b, 0, T) \end{array}$$

Relações on em $t = T$: $\text{on}(c, \top, T)$, $\text{on}(d, \top, T)$, $\text{on}(b, \top, T)$, $\text{on}(a, c, T)$.

Cláusulas unitárias: 1147 0, 1152 0, 1029 0, 1036 0, 1207 0, 1209 0, 1093 0, 1094 0.

4.1.3. Meta S_{f_1} ($T = 8$)

$$\begin{array}{ll} \text{at}(d, 3, T) \wedge \text{lev}(d, 0, T) & \text{at}(a, 4, T) \wedge \text{lev}(a, 1, T) \\ \text{at}(b, 5, T) \wedge \text{lev}(b, 1, T) & \text{at}(c, 4, T) \wedge \text{lev}(c, 2, T) \end{array}$$

Relações on em $t = T$: $\text{on}(d, \top, T)$, $\text{on}(a, d, T)$, $\text{on}(b, d, T)$, $\text{on}(c, a, T) \wedge \text{on}(c, b, T)$.
Aqui c está em ponte sobre a e b , que ficam lado a lado sobre d .

Cláusulas unitárias: 2236 0, 2237 0, 2061 0, 2064 0, 2121 0, 2123 0, 2179 0, 2182 0.

4.1.4. Meta S_{f_2} ($T = 9$)

$$\begin{array}{ll} \text{at}(d, 3, T) \wedge \text{lev}(d, 0, T) & \text{at}(c, 4, T) \wedge \text{lev}(c, 1, T) \\ \text{at}(a, 4, T) \wedge \text{lev}(a, 2, T) & \text{at}(b, 5, T) \wedge \text{lev}(b, 2, T) \end{array}$$

Relações on em $t = T$: $\text{on}(d, \top, T)$, $\text{on}(c, d, T)$, $\text{on}(a, c, T)$, $\text{on}(b, c, T)$.

Cláusulas unitárias: 2493 0, 2494 0, 2436 0, 2438 0, 2318 0, 2322 0, 2378 0, 2381 0.

4.1.5. Meta S_{f_3} ($T = 3$, como executado)

$$\begin{array}{ll} \text{at}(c, 0, T) \wedge \text{lev}(c, 0, T) & \text{at}(a, 2, T) \wedge \text{lev}(a, 0, T) \\ \text{at}(b, 5, T) \wedge \text{lev}(b, 0, T) & \text{at}(d, 0, T) \wedge \text{lev}(d, 1, T) \end{array}$$

Relações on em $t = T$: $\text{on}(c, \top, T)$, $\text{on}(a, \top, T)$, $\text{on}(b, \top, T)$, $\text{on}(d, c, T) \wedge \text{on}(d, a, T)$.
Aqui d está em ponte sobre c e a , sem slot vazio entre eles.

Cláusulas unitárias: 890 0, 895 0, 774 0, 778 0, 836 0, 837 0, 948 0, 953 0.

4.2. Situação 2

4.2.1. Estado inicial S_0

$$\begin{array}{ll} \text{at}(c, 0, 0) \wedge \text{lev}(c, 0, 0) & \text{at}(a, 0, 0) \wedge \text{lev}(a, 1, 0) \\ \text{at}(b, 1, 0) \wedge \text{lev}(b, 1, 0) & \text{at}(d, 3, 0) \wedge \text{lev}(d, 0, 0) \end{array}$$

Relações on em $t = 0$: $\text{on}(c, \top, 0)$, $\text{on}(d, \top, 0)$, $\text{on}(a, c, 0)$, $\text{on}(b, c, 0)$. O slot s_2 da mesa fica vazio entre c e d .

Cláusulas unitárias: 119 0, 124 0, 1 0, 8 0, 61 0, 67 0, 180 0, 181 0.

4.2.2. Meta S_5 ($T = 5$)

$$\begin{array}{ll} \text{at}(d, 3, T) \wedge \text{lev}(d, 0, T) & \text{at}(c, 4, T) \wedge \text{lev}(c, 1, T) \\ \text{at}(a, 4, T) \wedge \text{lev}(a, 2, T) & \text{at}(b, 5, T) \wedge \text{lev}(b, 2, T) \end{array}$$

Relações on em $t = T$: $\text{on}(d, \top, T)$, $\text{on}(c, d, T)$, $\text{on}(a, c, T)$, $\text{on}(b, c, T)$.

Cláusulas unitárias: 1465 0, 1466 0, 1408 0, 1410 0, 1290 0, 1294 0, 1350 0, 1353 0.

4.3. Situação 3

O estado inicial S_0 é o mesmo da Situação 1, com a mesma representação formal, relações on e cláusulas unitárias.

4.3.1. Meta S_7 ($T = 6$)

$$\begin{array}{ll} \text{at}(c, 0, T) \wedge \text{lev}(c, 0, T) & \text{at}(a, 0, T) \wedge \text{lev}(a, 1, T) \\ \text{at}(b, 1, T) \wedge \text{lev}(b, 1, T) & \text{at}(d, 3, T) \wedge \text{lev}(d, 0, T) \end{array}$$

Relações on em $t = T$: $\text{on}(c, \top, T)$, $\text{on}(d, \top, T)$, $\text{on}(a, c, T)$, $\text{on}(b, c, T)$.

Cláusulas unitárias: 1661 0, 1666 0, 1543 0, 1550 0, 1603 0, 1609 0, 1722 0, 1723 0.

4.4. Cláusulas dos grupos (Situação 1, $S_0 \rightarrow S_{f4}$, $T = 4$)

A Tabela 7 mostra um exemplo de cláusula de cada grupo, com a respectiva forma DI-MACS.

O CNF desse cenário tem 136 cláusulas unitárias: as 16 de estado inicial e meta, e 120 do grupo [7] que fixam como falsas as variáveis blk do nível máximo (por exemplo, -19 0).

Tabela 7. Exemplos de cláusulas por grupo.

Grupo	Cláusula	DIMACS
3. Unicidade de posição	$\neg \text{at}(a, 0, 0) \vee \neg \text{at}(a, 1, 0)$	-1 -2 0
4. Unicidade de nível	$\text{lev}(a, 0, 0) \vee \dots \vee \text{lev}(a, 3, 0)$	7 8 9 10 0
5. Exclusão horizontal	$\neg \text{cov}(a, 3, 0, 0) \vee \neg \text{cov}(b, 3, 0, 0)$	-36 -95 0
6. Estabilidade	$\neg \text{at}(d, 0, 0) \vee \neg \text{lev}(d, 1, 0) \vee \text{occ}(0, 0, 0) \vee \text{occ}(1, 0, 0)$	-177 -182 234 238 0
7. Clear	$\neg \text{clr}(d, 0) \vee \neg \text{blk}(d, 0, 0, 0)$	-185 -187 0
8. Pré-condição (topo livre)	$\neg \text{mv}(d, c, 0, 0) \vee \text{clr}(d, 0)$	-1362 185 0
8. Pré-condição (destino livre)	$\neg \text{mv}(d, c, 0, 0) \vee \neg \text{lev}(c, 0, 0) \vee \neg \text{cov}(a, 0, 1, 0)$	-1362 -124 -14 0
9. Efeito (posição)	$\neg \text{mv}(d, c, 0, 0) \vee \text{at}(d, 0, 1)$	-1362 434 0
9. Efeito (nível)	$\neg \text{mv}(d, c, 0, 0) \vee \neg \text{lev}(c, 0, 0) \vee \text{lev}(d, 1, 1)$	-1362 -124 439 0
10. Frame axiom	$\neg \text{at}(c, 0, 0) \vee \text{at}(c, 0, 1) \vee \text{mv}(c, \cdot, \cdot, 0)$	-119 376 1334 ... 1353 0
11. Ação única	$\neg \text{mv}(a, b, 0, 0) \vee \neg \text{mv}(a, b, 1, 0)$	-1286 -1287 0

Tabela 8. Resultados de execução dos cenários no miniSAT.

Cenário / Meta	Horizonte (T)	Variáveis	Cláusulas	Resultado
Situação 1: $S_0 \rightarrow S_{f4}$	4	1.621	27.077	SAT
Situação 1: $S_0 \rightarrow S_{f1}$	8	2.985	53.041	SAT
Situação 1: $S_0 \rightarrow S_{f2}$	9	3.326	59.532	SAT
Situação 1: $S_0 \rightarrow S_{f3}$	3	1.280	20.586	SAT
Situação 2: $S_0 \rightarrow S_5$	5	1.962	33.568	SAT
Situação 3: $S_0 \rightarrow S_7$	6	2.303	40.059	SAT

Tabela 9. Correspondência entre a descrição formal e o código.

Regra formal	Código Python (<code>bw2cnf_var.py</code>)
Blocos e comprimentos $\ell(b)$	<code>BLOCKS = {'a':1, 'b':1, 'c':2, 'd':3}</code>
Mesa T	<code>TABLE = 'T'</code>
Pontos (0..6) e slots (6)	<code>MAX_POINT = 6, slots = range(MAX_POINT)</code>
Níveis (0..3)	<code>MAX_LEVEL = 3, levels = range(MAX_LEVEL + 1)</code>
Posições válidas $p \in [0, 6 - \ell(b)]$	<code>valid_positions(b)</code>
Slots cobertos $\{p, \dots, p + \ell(b) - 1\}$	<code>span(b, p)</code>
$\text{Overlap}(b, p, y, q)$	<code>spans_overlap(b, p, y, q)</code>
$\text{at}(b, p, t)$	<code>at[(b, p, t)] = new_var()</code>
$\text{lev}(b, l, t)$	<code>lev[(b, l, t)] = new_var()</code>
$\text{clr}(b, t)$	<code>clr[(b, t)] = new_var()</code>
$\text{mv}(b, y, p, t)$	<code>mv[(b, y, p, t)] = new_var()</code>
$\text{cov}(b, i, l, t)$ (auxiliar)	<code>cov[(b, i, l, t)] = new_var()</code>
“algum bloco cobre i no nível l ” (auxiliar)	<code>occ[(i, l, t)] = new_var()</code>
“ b cobre i com algo acima” (auxiliar)	<code>blk[(b, i, l, t)] = new_var()</code>
Estados das Situações 1, 2 e 3	<code>dicionário SITUAÇÕES ({bloco: (ponto, nível)})</code>
$\text{On}(b, y, t)$ (derivado)	<code>função derivar_on no interpretar.py</code>

4.5. Tamanho e resultado de cada CNF

5. Mapeamento: Descrição Formal $\rightarrow$ Código Python

O gerador do CNF (`bw2cnf_var.py`) implementa as estruturas do domínio.

5.1. Domínio e variáveis

5.2. Grupos de cláusulas

5.3. Pontos que merecem atenção

- **Clear(y) do destino não é pré-condição.** O grupo [8] exige “slots de destino livres” (`add(-m, -cov[(z, i, ...)])`), então cabem vários blocos lado a lado sobre o mesmo apoio, como em S_{f1} e S_{f2} .
- **clr não tem frame axiom.** O grupo [7] o define exatamente em cada instante.
- **cov, occ e blk não vão para o .map.** Só at, lev, clr e mv entram.
- **On não é variável.** Ele é reconstruído de at, lev e sobreposição de spans.

5.4. Mudanças em relação ao caso simples (`bw2cnf.py`)

6. Execução Passo a Passo

Os três scripts (`bw2cnf_var.py`, `interpretar.py` e `validar_plano.py`) ficam na mesma pasta. A reprodução dos experimentos é feita pela linha de comando:

Tabela 10. Grupos de cláusulas e blocos do código.

Regra formal	Bloco no código
1. Estado inicial	# [1] Estado inicial
2. Meta	# [2] Meta
3. Unicidade de posição	# [3] Unicidade de posicao
4. Unicidade de nível	# [4] Unicidade de nivel
5. Exclusão horizontal	# [5] Exclusao horizontal
6. Estabilidade ($\lceil \ell/2 \rceil$ slots de apoio)	# [6] Estabilidade
7. Clear	# [7] Clear
8. Pré-condições de move	# [8] Pre-condicao: ...
9. Efeitos de move	# [9] Efeito: ...
10. Frame axioms	# [10] Frame axioms
11. Ação única por passo	# [11] Acao unica por passo
12. Ordem parcial $\varphi_1 \prec_P \varphi_2$	# [12] Ordem parcial e lista ORDEM_PARCIAL
Definição de cov e occ	# [Aux] definicao exata de cov / occ
Validade dos estados de entrada	função validar_estado

Tabela 11. Código do caso simples *versus* tamanho variável.

Caso simples	Caso de tamanho variável
on_var[(x, y, t)]	at[(b, p, t)] e lev[(b, l, t)]
move_var[(x, y, z, t)], ori- gem y e destino z	mv[(b, y, p, t)], destino qualitativo (y) e ponto p
(A)(B) um lugar por bloco	[3] e [4] unicidade de posição e de nível
(C) um bloco sobre cada bloco não existe	[5] exclusão horizontal por slot [6] estabilidade
(D) clear $\leftrightarrow$ nada sobre x	[7] clr por sobreposição de spans
(E) uma ação por passo	[11] ação única por passo
(F) pré-condições e efeitos	[8] pré-condições e [9] efeitos
(G1–G4) frame axioms de on e clear	[10] frame axioms só de at e lev
INITIAL_ON, GOAL_ON, HORIZON	SITUACOES e argumentos -situacao, -inicial, -meta, -horizonte

```

1 # 1. Gerar CNF e Mapeamento
2 python3 bw2cnf_var.py --situacao 1 --meta Sf4 --horizonte 4 --cnf sit1.
   cnf --map sit1.map
3
4 # 2. Executar o SAT Solver
5 minisat sit1.cnf resultado1.txt
6
7 # 3. Traduzir o Resultado para Portugues
8 python3 interpretar.py resultado1.txt --map sit1.map

```

Passo 1. Gera `sit1.cnf` (para o solver) e `sit1.map` (tradução número $\rightarrow$ símbolo). Se `-cnf` e `-map` forem omitidos, os nomes padrão são `trab01_blocos2SAT.cnf` e `trab01_blocos2SAT.map`.

Passo 2. A primeira linha do `resultado1.txt` é SAT ou UNSAT. Com UNSAT, aumenta-se o `-horizonte`.

Passo 3. Imprime o plano em português, o estado final e as relações on. Com `-verbose`, mostra o estado em cada instante.

6.1. Cenários

Tabela 12. Parâmetros de execução de cada cenário.

Situação	-meta	-horizonte	Saída
1	Sf1	8	resultado1.txt
1	Sf2	9	–
1	Sf3	3	–
1	Sf4	4	–
2	S5	5	resultado2.txt
3	S7	6	resultado3.txt

6.2. Observações

- O horizonte da tabela é o menor T com plano. Para provar que é mínimo, execute-se com $T - 1$ e confere-se o UNSAT.
- `-inicial` (por padrão S0) permite começar de outro estado.
- Cada execução sobrescreve o `.cnf` e o `.map` de mesmo nome, então devem-se usar nomes diferentes por cenário.

7. Interpretação da Saída do SAT Solver

7.1. O solver não conhece símbolos

O *miniSAT* é puramente booleano. Ele só conhece variáveis numeradas $1, 2, \dots, N$ e cláusulas sobre elas. Nomes como $mv(d, c, 0, 0)$ não existem para ele. A tradução número $\rightarrow$ símbolo é feita pelo arquivo `.map`.

7.2. Formato da saída

O arquivo `resultadoN.txt` tem o formato:

```
SAT
-1 -2 -3 4 -5 -6 7 -8 ... 0
```

- A primeira linha é SAT (existe plano com T passos) ou UNSAT (não existe).
- A segunda linha é o modelo: um inteiro por variável, terminado por 0.
- Inteiro positivo significa variável **verdadeira**. Negativo significa **falsa**.

7.3. O arquivo `.map`

Cada linha associa um número a um símbolo, e a primeira linha guarda o horizonte:

```
# HORIZON 4
1362 mv(d,c,0,0)
1375 mv(a,b,5,1)
180  at(d,3,0)
182  lev(d,1,0)
```

7.4. O que o `interpretar.py` faz

- **Leitura do mapa:** associa cada número ao seu símbolo.
- **Leitura do resultado:** se a primeira linha for UNSAT, informa que não há plano. Caso contrário, guarda os números positivos.
- **Filtragem:** separa os `mv` verdadeiros (as ações) e os `at` e `lev` verdadeiros (os estados).
- **Ordenação temporal:** ordena as ações por t e traduz cada uma para uma frase em português.
- **Estado final e on:** reconstrói `at` e `lev` no último instante e deriva as relações `on`.

7.5. Exemplo: Situação 1, $S_0 \rightarrow S_{f4}, T = 4$

As variáveis `mv` verdadeiras no modelo estão na Tabela 13.

Tabela 13. Variáveis `mv` verdadeiras no modelo.

Número	Símbolo	Leitura
1362	$mv(d, c, 0, 0)$	em $t = 0$, mover d para cima de c , em $p = 0$
1375	$mv(a, b, 5, 1)$	em $t = 1$, mover a para cima de b , em $p = 5$
1536	$mv(d, T, 2, 2)$	em $t = 2$, mover d para a mesa, em $p = 2$
1544	$mv(a, c, 0, 3)$	em $t = 3$, mover a para cima de c , em $p = 0$

Saída do interpretador:

```
PLANO ENCONTRADO (4 acoes):
1. t=0: mover bloco 'd' para CIMA de 'c' em p=0
2. t=1: mover bloco 'a' para CIMA de 'b' em p=5
3. t=2: mover bloco 'd' para a MESA em p=2
4. t=3: mover bloco 'a' para CIMA de 'c' em p=0

ESTADO FINAL (t=4):
```

```
a: p=0, nivel 1      b: p=5, nivel 0
c: p=0, nivel 0      d: p=2, nivel 0
```

RELACOES 'on' em t=4:

```
a esta sobre: c
b, c, d estao sobre: T
```

Os números dependem só do CNF, mas o modelo escolhido pode mudar de um solver para outro: o plano obtido com o *miniSAT* pode ser outro de mesmo tamanho.

7.6. Derivação de on

A relação `on` não é variável do CNF. Ela é reconstruída a partir do estado final: se $\text{lev}(b) = 0$, então $\text{on}(b, T)$; se $\text{lev}(b) = l > 0$, então $\text{on}(b, y)$ para todo bloco y com $\text{lev}(y) = l - 1$ cujo span sobrepõe o de b . Isso permite apoio múltiplo. Por exemplo, em S_0 , d em $p = 3$ no nível 1 fica “ d está sobre: a, b ” (a ponte).

7.7. Como ler UNSAT

UNSAT significa que **não existe plano com T passos ou menos**. Há duas causas:

- o horizonte é curto demais: aumenta-se T até dar SAT. O menor T satisfazível é o comprimento do plano mínimo (por exemplo, S_{f4} dá UNSAT com $T = 3$ e SAT com $T = 4$);
- o estado inicial ou a meta são fisicamente inválidos (sobreposição, instabilidade), e então nenhum T funciona.

Como o CNF permite no máximo uma ação por passo, um T maior que o mínimo continua satisfazível.

7.8. Comparação com o plano manual

O número de ações do plano do solver tem de coincidir com o do plano manual ótimo. As ações em si podem diferir, porque o mesmo estado final costuma ter vários planos de mesmo tamanho.

7.9. Correspondência entre `.cnf` e `.map`

Sem o `.map` correspondente ao `.cnf` que gerou o resultado, a saída do *miniSAT* é só uma lista de inteiros sem significado. Se outro cenário for gerado e sobrescrever o `.map`, a interpretação do resultado antigo fica errada. Por isso, deve-se usar um par `.cnf/.map` com nome próprio para cada cenário.

8. Conclusão

A abordagem via SAT solver mostrou-se viável para resolver o problema estendido do Mundo dos Blocos com restrições espaciais e mecânicas: todos os cenários avaliados resultaram em SAT, com instâncias de até 3.326 variáveis e 59.532 cláusulas. A codificação declarativa em CNF garante a otimalidade dos planos quando executada incrementalmente para $T = 0, 1, 2, \dots$ até a obtenção da primeira solução satisfatível (SAT).

Referências

- [1] Mota, E. (2026). *Manual de Resolução do Trabalho: Mundo dos Blocos de Tamanho Variável via SAT Solver*. Fundamentos de Inteligência Artificial, UFAM.
- [2] Mota, E. (2026). *Manual de Estudo: Mundo dos Blocos via SAT Solver*. Fundamentos de Inteligência Artificial, UFAM.
- [3] Eén, N. and Sörensson, N. (2003). An Extensible SAT-solver. In *Theory and Applications of Satisfiability Testing (SAT 2003)*, LNCS 2919, Springer.